GUILLAUME ARASSE TD2

SmartMove

Rapport de conception et schémas BD

1. Choix de PostgreSQL

1.1 Présentation

Pour stocker et interroger les données de mobilité du réseau Tisséo, j'ai choisi PostgreSQL comme système de gestion de base de données relationnelle.

1.2 Critères de sélection

Critère	PostgreSQL	Pourquoi c'est décisif pour SmartMove
Typage avancé	Types natifs : TIME, DATE, BOOLEAN, DOUBLE PRECISION, JSONB.	Les horaires GTFS sont stockés en TIME, les résultats d'appels LLM en JSONB (table requete_utilisateur), les jours de service en BOOLEAN.
Open source & gratuit	Licence PostgreSQL (similaire BSD) — aucun coût de licence.	Cohérent avec l'approche open data du projet (données Tisséo libres, OpenStreetMap, Python).

1.3 Comparaison avec les alternatives écartées

Alternative	Pourquoi envisagée
MySQL	Très répandu, bonne documentation.
SQLite	Simple à déployer, pas de serveur.
MongoDB	Flexible, stockage JSON natif.

2. Difficultés rencontrées

2.1 Horaires GTFS dépassant 24 heures

Le format GTFS autorise des horaires au-delà de minuit exprimés en dépassant 24h (ex. : 25:30:00 pour un trajet partant à 1h30 du matin le lendemain). PostgreSQL ne peut pas stocker une valeur de type TIME supérieure à 23:59:59, ce qui provoquait une erreur lors de l'import de stop_times.txt.

Solution : j'ai écrit la fonction convert_gtfs_time() qui applique un modulo 24 sur les heures avant insertion (25:30:00 → 01:30:00). Cette conversion a été validée sur l'ensemble du jeu de données avant de l'intégrer dans le pipeline d'import.

2.2 Volume des données et saturation mémoire

Le fichier stop_times.txt représente plusieurs milliers de lignes. Une insertion ligne par ligne avec cursor.execute() prenait plus de 40 minutes et finissait par saturer la RAM disponible, rendant le processus inutilisable en pratique.

Solution : passage à une insertion par lots de 500 lignes. Le temps d'import est passé à environ 8 minutes et la consommation mémoire est restée stable tout au long du chargement.

2.3 Désambiguïsation des arrêts dans le chatbot

Plusieurs arrêts du réseau Tisséo partagent un même nom ou un nom très proche (ex. : plusieurs arrêts « Capitole » sur des lignes différentes, ou « Jean Jaurès » qui désigne à la fois une station de métro et plusieurs arrêts de bus). Lorsque l'utilisateur tapait un nom ambigu, le LLM choisissait arbitrairement l'un d'eux, ce qui pouvait produire un itinéraire incorrect ou impossible.

Solution : mise en place d'un mécanisme de désambiguïsation côté backend. Lorsqu'une requête retourne plusieurs arrêts pour un même nom, le système suspend le calcul d'itinéraire et renvoie au frontend la liste des correspondances avec leurs lignes associées. L'utilisateur sélectionne l'arrêt souhaité et la conversation reprend avec le bon identifiant.

3. Schéma relationnel de la base de données

La base de données SmartMove comprend 10 tables métier, 1 table de traçabilité des requêtes utilisateurs, et 2 vues SQL. Le schéma ci-dessous détaille pour chaque table l'ensemble des colonnes, leur type SQL et leurs contraintes d'intégrité. Les clés primaires sont indiquées en bleu (PK), les clés étrangères en orange (FK).

GUILLAUME ARASSE TD2

Table	Colonne	Type SQL	Contrainte
agence	**agence_id**	*VARCHAR(50)*	**PK e**
	nom	*VARCHAR(200)*	NOT NULL
	url	*TEXT*	
	timezone	*VARCHAR(50)*	DEFAULT 'Europe/Paris'
	langue	*VARCHAR(10)*	DEFAULT 'fr'
	telephone	*VARCHAR(30)*	Nullable
	email	*VARCHAR(100)*	Nullable

Table	Colonne	Type SQL	Contrainte
ligne	**ligne_id**	*VARCHAR(50)*	**PK**
	agence_id	*VARCHAR(50)*	**FK → agence(agence_id)**
	code_ligne	*VARCHAR(20)*	
	nom_long	*VARCHAR(200)*	
	description	*TEXT*	Nullable
	type_transport	*INTEGER*	0=tramway 1=métro 2=train 3=bus
	couleur	*VARCHAR(6)*	Hex sans # — ex. D4003B
	couleur_texte	*VARCHAR(6)*	
	url	*TEXT*	

Table	Colonne	Type SQL	Contrainte
arret	**arret_id**	*VARCHAR(50)*	**PK**
	code_arret	*VARCHAR(20)*	Code affiché aux poteaux
	nom	*VARCHAR(200)*	NOT NULL
	description	*TEXT*	
	latitude	*DOUBLE PRECISION*	Coordonnée WGS84
	longitude	*DOUBLE PRECISION*	Coordonnée WGS84
	type_lieu	*INTEGER*	0=arrêt 1=station 2=entrée
	arret_parent_id	*VARCHAR(50)*	**FK**
	zone_id	*VARCHAR(20)*	Zone tarifaire

	url	*TEXT*	
	ville	*VARCHAR(100)*	
	accessibilite_fauteuil	*INTEGER*	0=inconnu 1=accessible 2=non accessible

Table	Colonne	Type SQL	Contrainte
calendrier	**service_id**	*VARCHAR(50)*	**PK**
	lundi	*BOOLEAN*	
	mardi	*BOOLEAN*	
	mercredi	*BOOLEAN*	
	jeudi	*BOOLEAN*	
	vendredi	*BOOLEAN*	
	samedi	*BOOLEAN*	
	dimanche	*BOOLEAN*	
	date_debut	*DATE*	Début de validité du service
	date_fin	*DATE*	Fin de validité du service

Table	Colonne	Type SQL	Contrainte
calendrier_exception	**exception_id**	*SERIAL*	**PK**
	service_id	*VARCHAR(50)*	**FK → calendrier(service_id)**
	date	*DATE*	Date concernée par l'exception
	type_exception	*INTEGER*	1=ajout de service 2=suppression

GUILLAUME ARASSE TD2

Table	Colonne	Type SQL	Contrainte
arret_trajet	**arret_trajet_id**	*SERIAL*	**PK**
	trajet_id	*VARCHAR(50)*	**FK → trajet(trajet_id)**
	arret_id	*VARCHAR(50)*	**FK → arret(arret_id)**
	heure_arrivee	*TIME*	Horaire GTFS converti (modulo 24h)
	heure_depart	*TIME*	
	ordre_arret	*INTEGER*	
	texte_arret	*VARCHAR(200)*	Texte affiché à bord
	type_prise_en_charge	*INTEGER*	0=régulier 1=sur demande
	type_depose	*INTEGER*	0=régulier 1=sur demande
	distance_parcourue	*DOUBLE PRECISION*	Distance cumulée depuis le départ (km)

Table	Colonne	Type SQL	Contrainte
frequence	**frequence_id**	*SERIAL*	**PK**
	trajet_id	*VARCHAR(50)*	**FK → trajet(trajet_id)**
	heure_debut	*TIME*	Début de la plage de fréquence
	heure_fin	*TIME*	Fin de la plage de fréquence
	intervalle_secondes	*INTEGER*	Fréquence de passage en secondes
	type_exactitude	*INTEGER*	0=approximatif 1=exact

Table	Colonne	Type SQL	Contrainte
horaire	**horaire_id**	*SERIAL*	**PK**
	ligne_id	*VARCHAR(50)*	**FK → ligne(ligne_id)**
	arret_id	*VARCHAR(50)*	**FK → arret(arret_id)**
	trajet_id	*VARCHAR(50)*	**FK → trajet(trajet_id)**
	service_id	*VARCHAR(50)*	**FK → calendrier(service_id)**
	heure_passage	*TIME*	Horaire de passage à l'arrêt
	direction	*INTEGER*	0 ou 1 — sens de circulation
	jour_semaine	*VARCHAR(20)*	Ex. : lundi, samedi
	date_validite	*DATE*	

	created_at	*TIMESTAMP*	DEFAULT NOW()

GUILLAUME ARASSE TD2

Table	Colonne	Type SQL	Contrainte
evenement	**evenement_id**	*SERIAL*	**PK**
	titre	*VARCHAR(300)*	NOT NULL
	description	*TEXT*	
	lieu	*VARCHAR(300)*	
	arret_proche_id	*VARCHAR(50)*	**FK → arret(arret_id)**
	date_debut	*TIMESTAMP*	
	date_fin	*TIMESTAMP*	
	url	*TEXT*	
	categorie	*VARCHAR(100)*	Ex. : concert, sport, foire
	created_at	*TIMESTAMP*	DEFAULT NOW()

Table	Colonne	Type SQL	Contrainte
actualite	**actualite_id**	*SERIAL*	**PK**
	titre	*VARCHAR(500)*	NOT NULL
	contenu	*TEXT*	
	resume	*TEXT*	Résumé généré par LLM
	source	*VARCHAR(200)*	
	url	*TEXT*	
	date_publication	*TIMESTAMP*	
	categorie	*VARCHAR(100)*	travaux / perturbation / événement / info
	lignes_concernees	*VARCHAR(200)*	Ex. : L1, T2, 38
	created_at	*TIMESTAMP*	DEFAULT NOW()
	updated_at	*TIMESTAMP*	Mis à jour automatiquement

Table	Colonne	Type SQL	Contrainte
requete_utilisateur	**requete_id**	*SERIAL*	**PK**
	question	*TEXT*	Message brut de l'utilisateur
	requete_sql	*TEXT*	Requête SQL générée par le LLM
	resultat_json	*JSONB*	Résultat retourné au frontend

	satisfaction	*INTEGER*	1=positif 0=négatif — feedback utilisateur
	created_at	*TIMESTAMP*	DEFAULT NOW()

Récapitulatif des relations (clés étrangères)

Table source → colonne FK	Table cible → colonne PK	Signification
ligne → agence_id	agence → agence_id	Une ligne appartient à une agence
trajet → ligne_id	ligne → ligne_id	Un trajet appartient à une ligne
trajet → service_id	calendrier → service_id	Un trajet suit un calendrier de service
arret → arret_parent_id	arret → arret_id	Auto-référence : un arrêt peut avoir une station mère
arret_trajet → trajet_id	trajet → trajet_id	Séquence d'arrêts d'un trajet
arret_trajet → arret_id	arret → arret_id	Arrêt desservi dans ce trajet
frequence → trajet_id	trajet → trajet_id	Plages de fréquence d'un trajet
horaire → ligne_id	ligne → ligne_id	Horaire lié à une ligne
horaire → arret_id	arret → arret_id	Horaire lié à un arrêt
horaire → trajet_id	trajet → trajet_id	Horaire lié à un trajet
horaire → service_id	calendrier → service_id	Horaire lié à un service calendaire
calendrier_exception → service_id	calendrier → service_id	Exception (ajout/suppression) sur un service
evenement → arret_proche_id	arret → arret_id	Arrêt de transport le plus proche de l'événement